

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Brno, Fall 2025

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Advisor: RNDr. Martin Ukrop, Ph.D.

Consultant: Ing. Jan Štourač

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

The following AI tools were used during the preparation of this thesis:

- **Claude Code** (Anthropic) for assistance with writing and debugging the implementation code.
- **Claude 4.5 Opus** (Anthropic) for improving grammar, rephrasing, and assistance with styling of the thesis text.

I have reviewed all AI-generated content and take full responsibility for the final text and code presented in this thesis.

Adam David Malý

Advisor: RNDr. Martin Ukrop, Ph.D.
Consultant: Ing. Jan Štourač

Abstract

Red Hat OpenShift AI is an enterprise machine-learning platform whose workbenches — containerised development environments for data scientists — are currently managed either through a web dashboard or by directly manipulating Kubernetes resources. Both approaches are time-consuming or require significant platform expertise. This thesis explores whether workbench management can instead be driven entirely through natural language by connecting an AI agent to the platform via the Model Context Protocol (MCP), an open standard for integrating AI applications with external tools and data sources. The result is an MCP server that lets users create, configure, monitor, and delete workbenches simply by describing what they need to an AI assistant. The server operates in a safe read-only mode by default and is validated through unit tests and automated evaluation of end-to-end agent behaviour. Because the platform is transitioning to a redesigned workbench architecture (Kubeflow Notebooks 2.0), the implementation serves as a proof of concept that demonstrates the feasibility of the approach and establishes patterns for a future production-ready server. The source code is released as open source under the Apache License 2.0.

Keywords

MCP, AI, Agents, RHOAI, Workbench, Server, Kubernetes, Kubeflow

Contents

1	Introduction	1
2	Background: Artificial Intelligence and the Model Context Protocol	3
2.1	Large Language Models and AI Assistants	3
2.2	AI Agents and Tool Use	4
2.3	The Model Context Protocol	5
2.3.1	Protocol Overview	5
2.3.2	MCP Components	5
2.3.3	Client Capabilities	6
2.3.4	Server Capabilities	8
2.3.5	Transport Mechanisms	8
3	Background: Red Hat OpenShift AI and Workbenches	10
3.1	Kubernetes and OpenShift	10
3.1.1	Kubernetes Architecture	11
3.1.2	Red Hat OpenShift	12
3.2	Kubeflow, Open Data Hub, and Red Hat OpenShift AI .	14
3.2.1	Kubeflow	14
3.2.2	Open Data Hub	14
3.2.3	Red Hat OpenShift AI	15
3.2.4	Hardware Profiles	15
3.3	Kubeflow Notebooks and Workbenches	15
3.3.1	The Kubeflow Notebook CRD	16
3.3.2	RHOAI Workbenches	16
3.3.3	Notebook Images	17
3.3.4	The Current Management Workflow	17
4	Analysis and Design	18
4.1	Motivation and Goals	18
4.2	Requirements	19
4.2.1	Functional Requirements	20
4.2.2	Non-Functional Requirements	21
4.3	Technology Choices	22
4.4	Architecture	24
4.4.1	Project Structure	24

4.4.2	Two-Mode Design	25
4.4.3	Kubernetes Client Strategy	25
4.4.4	Build Pipeline	26
5	Implementation	27
5.1	MCP Tools	27
5.1.1	Workbench Management	27
5.1.2	Hardware Profiles	29
5.1.3	Notebook Images	30
5.1.4	Storage and Infrastructure	31
5.1.5	Resource Consumption	31
5.2	MCP Resources and Prompts	32
5.2.1	Image Catalog Resource	32
5.2.2	Hardware Profiles Resource	33
5.2.3	Namespaces Resource	33
5.2.4	Create Workbench Prompt	33
5.3	Security	34
5.4	Code Quality	35
5.4.1	Static Analysis	35
5.4.2	Unit Testing	35
5.5	Evaluation	36
5.5.1	Evaluation Setup	37
5.5.2	Test Cases	37
5.5.3	Running the Evaluation	38
5.5.4	Results and Learnings	38
6	Conclusion	40
6.1	Future Work: Notebooks 2.0	41
	Attachments	43
	Bibliography	44

List of Tables

5.1	Workbench management tools.	28
5.2	Hardware profile tools.	29
5.3	Notebook image tools.	30
5.4	Storage and infrastructure tools.	31
5.5	Resource consumption monitoring tools (all read-only). .	32

List of Figures

2.1	MCP architecture	7
3.1	The platform stack	10

1 Introduction

Large language models (LLMs) have rapidly evolved in recent years from conversational assistants into capable agents that can reason, plan, and take actions in the real world (1, 2). This shift has been made possible by the ability to give LLMs access to external *tools* — functions that let a model query a database, call an API, or manage infrastructure, all in response to a natural language request (Chapter 2 describes this agentic paradigm in detail). As the need for such integrations grows, a key challenge emerges: how do different AI hosts and models connect to the ever-expanding set of tools and data sources in a consistent way?

Anthropic introduced the Model Context Protocol (MCP) in November 2024 (3) to address exactly this problem. MCP is an open standard that defines how AI applications connect to external tools and data sources through a client-server architecture (see Section 2.3). Instead of each AI host maintaining its own integrations, any MCP-compatible host (Cursor, Claude Desktop, or a custom agent) can connect to any MCP server and immediately gain access to the capabilities it exposes. This has already led to a growing ecosystem of MCP servers for services ranging from databases and file systems to cloud APIs and developer tools.

Red Hat OpenShift AI (RHOAI) (4) is an enterprise machine learning platform built on top of Red Hat OpenShift (5), which is based on Kubernetes (6). One of its core features are workbenches — containerised JupyterLab (7) environments that data scientists and engineers use for model training and experimentation (Chapter 3 explains the platform architecture). Managing clusters with many workbench instances can be a tedious and error-prone task, currently requiring navigation of the RHOAI web UI or direct manipulation of Kubernetes resources. The goal of this thesis is to eliminate that problem by allowing users to manage their workbenches entirely through natural language — simply describing what they want to an AI agent, which then carries out the necessary operations on their behalf.

To achieve this, the thesis presents the design and implementation of an MCP server that connects AI agents to RHOAI (Chapter 4 derives the requirements and architecture; Chapter 5 describes the

implementation). The server covers the full workbench management lifecycle — from creation and configuration to monitoring and deletion — and operates in a safe read-only mode by default to prevent accidental modifications. It also explores all three capability types that the MCP standard offers (tools, resources, and prompts), demonstrating how they can work together in practice. Because RHOAI is currently transitioning to a redesigned workbench architecture (Kubeflow Notebooks 2.0 (8)), the implementation serves as a proof of concept: it validates the feasibility of the approach and establishes the architectural patterns and evaluation methodology that a future production-ready server can build upon (Chapter 6 discusses this future work). The complete source code is available as open source under the Apache License 2.0 at <https://github.com/amaly/mcp-server-rhoai>.

2 Background: Artificial Intelligence and the Model Context Protocol

This chapter introduces large language models and the agentic paradigm of tool use, then describes the Model Context Protocol in detail.

2.1 Large Language Models and AI Assistants

At the core of modern AI assistants are *large language models* (LLMs) — neural networks trained to predict and generate text by learning statistical patterns from vast amounts of data. The dominant architectural foundation for LLMs is the *Transformer*, introduced by Vaswani et al. in 2017 (9). The Transformer’s self-attention mechanism allows the model to relate every token in a sequence to every other token, enabling it to capture long-range dependencies in text far more effectively than earlier recurrent architectures. Another benefit of the Transformer is that it is parallelizable, making it possible to train large models on extensive datasets in a reasonable time.

Modern LLMs are trained in two main stages. In the first stage, *pre-training*, the model is exposed to hundreds of billions of tokens of text and learns to predict the next token in a sequence. This process, while deceptively simple, causes the model to internalise broad world knowledge, reasoning patterns, and linguistic structure. In the second stage, the pre-trained model is fine-tuned to behave as a helpful assistant, typically through a combination of supervised instruction fine-tuning and reinforcement learning from human feedback (RLHF) (10). The result is a model that responds to natural-language instructions in a coherent and useful way.

The products of this pipeline — systems such as OpenAI’s ChatGPT (11), Anthropic’s Claude (12), and Google’s Gemini (13) — are commonly referred to as *AI assistants*. An AI assistant takes a user’s message as input and produces a textual response. It is fundamentally *stateless* and *passive*: it answers questions, drafts documents, and explains concepts, but it does not take actions in the world, does not maintain persistent memory across sessions (by default), and cannot

access information beyond its training data or the content of the current conversation. This passivity is the key limitation that the agentic paradigm, described in the next section, overcomes.

2.2 AI Agents and Tool Use

Despite their broad capabilities, pure language models are isolated from the external world — their knowledge does not extend beyond their training data, they cannot query other systems, and they cannot take actions. The concept of an *AI agent* overcomes these limitations by equipping a language model with *tools* — callable functions that the model can invoke to interact with external systems, retrieve fresh information, or modify state. Agents can also be given *memory* — the ability to retain information across steps or sessions (14). Memory can range from simply keeping the full conversation history in the model’s context window to reading and writing records in an external database via dedicated memory tools.

The key mechanism enabling agent behaviour is *tool calling* (also known as *function calling*). When a model is given a set of tool definitions alongside a user request, it can — instead of producing a plain text response — output a structured call to one of those tools, specifying the tool name and the required arguments. The host application intercepts this call, executes the corresponding function, and feeds the result back into the model’s context, after which the model continues its reasoning. This loop can repeat multiple times within a single interaction, allowing the model to chain tool calls to complete complex, multi-step tasks.

Yao et al. formalised this pattern in *ReAct* (2), which unifies *reasoning* and *acting* into a single loop: the model first produces a chain-of-thought reasoning trace — thinking through what it knows and what it needs — and then selects and executes an action (a tool call). The output returned by the tool becomes part of the context for the next reasoning step. This structured loop gives agents a degree of *autonomy*: given a high-level goal, an agent can decompose it into sub-tasks, select appropriate tools, and adapt its behaviour based on the results it receives.

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

The practical impact of tool use is significant. An agent equipped with the right tools can list resources in a Kubernetes cluster, query resource consumption metrics, and delete workbench instances — all in response to a single natural-language instruction such as “delete workbenches running for more than a week”. The Model Context Protocol provides a standardised way of exposing those capabilities to any AI host.

2.3 The Model Context Protocol

The Model Context Protocol (MCP) is an open standard that defines how AI applications connect to external tools and data sources. This section describes the protocol’s motivation and architecture (Section 2.3.1), its three main components (Section 2.3.2), the capabilities exposed by clients (Section 2.3.3) and servers (Section 2.3.4), and the available transport mechanisms (Section 2.3.5).

2.3.1 Protocol Overview

MCP was introduced by Anthropic in November 2024 (3) to address a fundamental integration challenge. Its core motivation is the $M \times N$ *integration problem*: without a shared standard, connecting M AI hosts to N external services requires up to $M \times N$ custom integrations, each with its own authentication, serialisation, and error-handling logic. MCP reduces this to $M + N$ by providing a single protocol that both sides implement once.

The protocol is *transport-agnostic* and message-based, built on top of JSON-RPC 2.0 (15). Official SDKs are available for Python, TypeScript, Go, Java, Kotlin, and C# (16).

2.3.2 MCP Components

The MCP architecture defines three main components: *hosts*, *clients*, and *servers*.

A **host** is the AI application that the end user interacts with directly — for example Claude Desktop, Cursor, or a custom agent application. The host is responsible for managing the user interaction, sending

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

messages to the LLM, orchestrating tool calls, and processing the output of the LLM.

A **client** is a component that lives inside the host and manages exactly one connection to one MCP server. A single host can run multiple clients simultaneously, each connected to a different server, giving the agent access to multiple sets of tools at once. The client speaks the MCP protocol on behalf of the host — it sends requests to the server, receives responses and notifications, and forwards results back to the LLM context. Beyond handling requests, clients can also expose their own capabilities to servers, enabling server-initiated interactions described in Section 2.3.3.

A **server** is a lightweight process that exposes capabilities (tools, resources, and prompts) to clients (17). A server typically wraps one specific external system — a database, a cloud API, a file system, or a cloud platform. It contains the actual implementation of each capability. When a tool is invoked, the server executes the corresponding logic and returns the result. The server does not communicate with the LLM directly — it only communicates with the client that is connected to it. This separation of roles means that server authors do not need to know anything about which LLM or host will use their tools and other capabilities. The server-exposed primitives are described in Section 2.3.4. Figure 2.1 illustrates these relationships.

2.3.3 Client Capabilities

While servers expose tools, resources, and prompts, clients can expose their own capabilities back to servers, allowing servers to initiate certain interactions rather than only responding to them. MCP defines three such client capabilities (17).

Sampling lets a server ask the client to generate an LLM completion on its behalf. This enables servers to perform agentic sub-tasks — such as summarising a result or deciding a next step — without needing direct access to any language model.

Roots allow a server to query the client about the filesystem or URI boundaries it is permitted to operate in, so that servers respect the scope defined by the host environment.

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

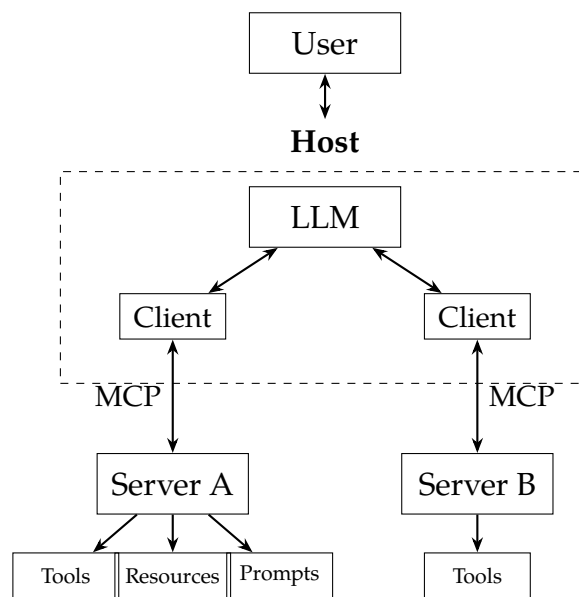


Figure 2.1: MCP architecture: a host application contains the LLM and one or more clients, each connected to exactly one server via the MCP protocol. Servers expose tools, resources, and prompts to the clients.

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

Elicitation lets a server request additional information from the user through the host, for example asking for a confirmation or a missing parameter before proceeding with an operation.

2.3.4 Server Capabilities

MCP servers expose capabilities to clients through three primitive types: tools, resources, and prompts (17).

Tools are the primary mechanism for giving an AI agent the ability to act. Each tool has a name, a human-readable description, and a JSON Schema that describes its input parameters. When the LLM decides that a tool is needed to fulfil a request, it emits a structured tool call containing the tool name and arguments. The host executes the call via the client and returns the result to the model's context. Tools are *model-initiated* — it is the LLM that decides when and how to invoke them.

Resources are URI-addressed data that the host can read and inject into the model's context. Unlike tools, resources are *host-initiated* — it is the host or the user that decides which resources to include, not the model. While conceptually similar to Retrieval-Augmented Generation (RAG) (18), where relevant documents are retrieved and injected into the prompt, MCP resources differ in that they are explicitly declared by the server and selected by the host or user, rather than retrieved automatically through a similarity search.

Prompts are pre-defined, parameterisable message templates that guide the model's behaviour for a specific task. A prompt can be invoked by the user or the host and expands into one or more messages that are injected into the conversation.

2.3.5 Transport Mechanisms

MCP is transport-agnostic (the protocol defines the structure of messages but not how they are physically carried between endpoints) and defines two standard transport mechanisms (17) — these are the transports supported by the official SDKs (16) and used by the ecosystem in practice. All messages follow the JSON-RPC 2.0 format (15), a lightweight remote procedure call protocol in which each request and

2. BACKGROUND: ARTIFICIAL INTELLIGENCE AND THE MODEL CONTEXT PROTOCOL

response is a single JSON object identified by a method name and a unique request identifier.

Standard I/O (stdio) is the simplest transport. The host launches the MCP server as a child process and communicates with it over the process's standard input and standard output streams. Each JSON-RPC message is a newline-delimited JSON object. This transport requires no network configuration, has no authentication overhead, and is inherently scoped to the local machine, making it well-suited for locally-installed tools.

HTTP with Server-Sent Events (SSE) is the remote transport. The client sends requests to the server via HTTP POST, and the server delivers responses and unsolicited notifications back via an SSE stream. This allows MCP servers to be deployed as network services, enabling multi-user scenarios and remote infrastructure management. A newer *Streamable HTTP* transport, which consolidates both directions into a single HTTP endpoint, is also defined in the specification as the recommended transport for new remote deployments (17).

3 Background: Red Hat OpenShift AI and Workbenches

This chapter describes the fundamental concepts of Red Hat OpenShift AI (RHOAI) and workbenches. These concepts provide the foundation for the MCP server implementation and the architecture choices discussed in later chapters. The sections follow the layered architecture from bottom to top: first OpenShift (based on Kubernetes) as the infrastructure layer (Section 3.1), then RHOAI (based on Kubeflow) as the machine-learning platform (Section 3.2), and finally workbenches (based on Kubeflow notebooks) as the user-facing JupyterLab environments (Section 3.3). Figure 3.1 illustrates this layered architecture.

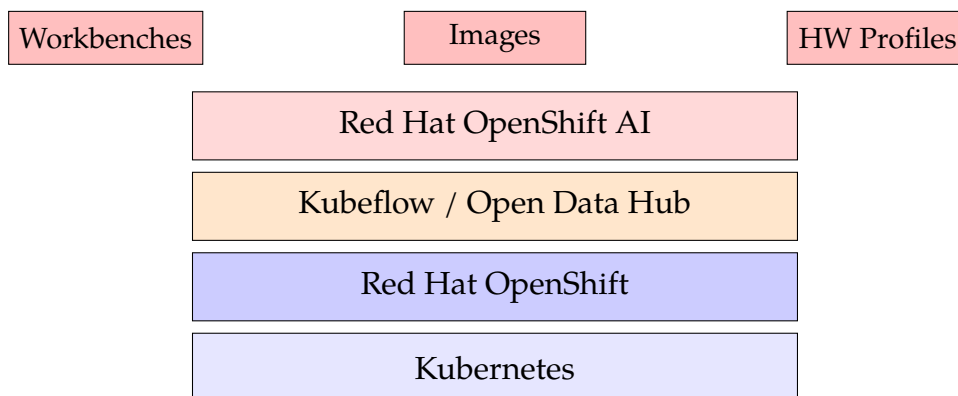


Figure 3.1: The platform stack: Kubernetes provides container orchestration, OpenShift adds enterprise features, Kubeflow/ODH provides ML components, and RHOAI is the commercial ML platform with workbenches, notebook images, and hardware profiles.

3.1 Kubernetes and OpenShift

This section introduces the two platform layers that underpin RHOAI: Kubernetes as the open-source orchestration engine (Subsection 3.1.1), and Red Hat OpenShift as the commercial distribution that extends it with enterprise features (Subsection 3.1.2).

3.1.1 Kubernetes Architecture

Kubernetes is an open-source platform for automating the deployment, scaling, and management of containerised applications (6). It was originally designed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF) (19). Kubernetes provides a declarative configuration model: users describe the *desired state* of their applications in YAML or JSON manifests, and the platform's control loop continuously reconciles the actual state of the cluster to match. This section introduces the core Kubernetes architectural concepts that are relevant to workbench management.

Nodes and the control plane. A Kubernetes cluster consists of a *control plane* and a set of worker machines called *nodes*. The control plane runs the API server, scheduler, and controller manager, which together accept user requests, decide where workloads should run, and ensure the desired state is maintained. Each worker node runs a kubelet agent that manages the *Pods* (the smallest deployable units, described below) scheduled on that node and reports status back to the control plane (6).

Pods. The smallest deployable unit in Kubernetes is a *pod* — a group of one or more containers that share the same network namespace and storage (20). In practice, most pods contain a single application container. Every workbench in Red Hat OpenShift AI runs as a pod.

Namespaces. Namespaces provide a mechanism for isolating groups of resources within a single cluster (21). In the context of Red Hat OpenShift AI, each data-science project maps to a Kubernetes namespace.

Persistent Volumes and Persistent Volume Claims. Containers are ephemeral by default: any data written inside a container is lost when the pod is terminated. To address this limitation, Kubernetes introduces a two-layer storage abstraction (22). A *PersistentVolume* (PV) represents a piece of storage provisioned in the cluster. A *PersistentVolumeClaim* (PVC) is a user's request for storage; when a PVC is created,

Kubernetes binds it to a suitable PV. Workbenches mount a PVC to persist notebooks and data across restarts.

StatefulSets. A *StatefulSet* manages a set of pods with stable network identities and persistent storage (23). Unlike a Deployment, which treats all pods as interchangeable, a StatefulSet guarantees that each pod retains its identity and its associated PVC across rescheduling. Workbenches are backed by StatefulSets, which ensures that a user's notebook environment is preserved even if the underlying node changes.

Custom Resource Definitions. Beyond these built-in resources, one of Kubernetes' most powerful extensibility features is the ability to define *Custom Resource Definitions* (CRDs) (24). A CRD registers a new resource type with the Kubernetes API server, allowing users and operators to manage domain-specific objects — such as machine learning notebooks — with the same tools and workflows used for built-in resources like pods or services. The Kubeflow project defines a Notebook CRD (API group `kubeflow.org/v1`) that represents a single notebook environment.

Labels and annotations. Additionally, Kubernetes objects carry two kinds of key-value metadata (25). *Labels* are used for selection and filtering — for example, RHOAI marks all notebook ImageStreams with the label `opendatahub.io/notebook-image=true` so that they can be distinguished from other ImageStreams in the namespace that are not notebook images. *Annotations* store non-identifying metadata such as display names, image version hashes, or hardware profile references. In RHOAI, workbench status, the selected notebook image, and the hardware profile are all stored as annotations on the workbench resource, as described in Section 3.3.

3.1.2 Red Hat OpenShift

Red Hat OpenShift is a commercial Kubernetes distribution developed by Red Hat (26). It packages upstream Kubernetes with a curated set of additional components and wraps them in enterprise support, cer-

tified operators, and a hardened security model. While any standard Kubernetes cluster can run containerised workloads, OpenShift adds several features that are particularly relevant to the platform stack discussed in this thesis.

Projects. OpenShift introduces the concept of a *project*, which is a thin wrapper around a Kubernetes namespace that adds default role bindings, network policies, and resource quotas (27). In practice, every OpenShift project maps one-to-one to a namespace, and the terms are often used interchangeably. Red Hat OpenShift AI uses projects to represent data-science teams.

ImageStreams. Standard Kubernetes references container images by their full registry URL. OpenShift extends this model with *ImageStreams* (28), which are namespace-scoped objects that track one or more tagged versions of a container image. An ImageStream decouples workloads from specific registry URLs: when a new image version is pushed, the ImageStream is updated and can automatically trigger redeployments. Red Hat OpenShift AI uses ImageStreams (API group `image.openshift.io/v1`) to manage the catalogue of notebook images available for workbenches.

Routes and built-in OAuth. OpenShift provides *Routes* as a native mechanism for exposing HTTP and HTTPS services to external traffic (29), fulfilling a role similar to Kubernetes Ingress but with tighter integration into the platform. Each workbench is exposed through a Route, which gives it a stable URL. OpenShift also ships a built-in OAuth server that handles user authentication (30). Workbenches leverage this to ensure that only authenticated users can access their JupyterLab environments.

Operators and the Operator Lifecycle Manager. OpenShift uses the *Operator* pattern extensively to manage complex software on top of the cluster (31). An Operator is a Kubernetes controller that watches custom resources and reconciles them toward their desired state. The Operator Lifecycle Manager (OLM) handles installation, upgrades, and dependency resolution for Operators. Red Hat OpenShift AI itself

is installed as an Operator, which in turn deploys and manages the KubeFlow components, dashboard, and other services that make up the platform.

3.2 KubeFlow, Open Data Hub, and Red Hat OpenShift AI

This section introduces the software layers that sit on top of OpenShift and together provide the machine-learning platform: KubeFlow as the upstream open-source framework (Section 3.2.1), Open Data Hub as the community project that integrates KubeFlow with OpenShift (Section 3.2.2), Red Hat OpenShift AI as its commercial counterpart (Section 3.2.3), and hardware profiles as the mechanism for defining compute-resource constraints (Section 3.2.4).

3.2.1 KubeFlow

KubeFlow is an open-source project that aims to make deployments of machine learning workflows on Kubernetes simple, portable, and scalable (32). Rather than being a single monolithic application, KubeFlow is a collection of components — each addressing a different stage of the ML lifecycle — that are deployed as Operators and custom resources on a Kubernetes cluster. Key components include KubeFlow Pipelines for orchestrating multi-step training workflows, KServe for model serving, Katib for hyperparameter tuning, and KubeFlow Notebooks for interactive development environments.

The most important KubeFlow component for workbench management is **KubeFlow Notebooks**, which provides a safe, scalable place for data scientists to write code. The Notebook CRD, its controller, and its relationship to RHOAI workbenches are discussed in detail in Section 3.3.

3.2.2 Open Data Hub

Open Data Hub (ODH) is an open-source community project that integrates upstream KubeFlow components with OpenShift-specific features (33). ODH provides a dashboard, a notebook controller, model

serving, and pipeline infrastructure — all deployed as Operators on an OpenShift cluster. Many of the custom resource API groups (such as `opendatahub.io`) originate from ODH. The CRDs, labels, and annotations defined by ODH form the foundation on which the commercial platform described in the next subsection is built.

3.2.3 Red Hat OpenShift AI

Red Hat OpenShift AI (RHOAI) is Red Hat’s commercial machine-learning platform derived from Open Data Hub and built on top of OpenShift (4). It packages the same core components as ODH but adds enterprise hardening, certified Operators, and commercial support. RHOAI is installed on an OpenShift cluster as an Operator and provides a web-based *dashboard* — a graphical interface from which users can create data-science projects, launch and manage workbenches, configure notebook images and hardware profiles, and monitor resource consumption without writing any YAML or using the command line.

3.2.4 Hardware Profiles

A *hardware profile* is an RHOAI-specific custom resource that defines a named set of compute-resource constraints for a workbench. Each profile specifies one or more resource identifiers — typically CPU, memory, and optionally a GPU (referred to as an *accelerator* in RHOAI) — with a default, minimum, and maximum count for each. When a user creates a workbench, they select a hardware profile. Platform then translates the profile’s resource definitions into Kubernetes resource requests and limits on the workbench’s pod. Administrators can create hardware profiles to standardise the compute tiers available to data scientists (for example, “Small” with 2 CPUs and 4 GiB of memory, or “GPU Large” with 8 CPUs, 32 GiB, and one NVIDIA GPU).

3.3 Kubeflow Notebooks and Workbenches

This section describes the topmost layer of the platform stack: the interactive development environments that data scientists use daily. The upstream Kubeflow project calls them *Notebooks*; Red Hat OpenShift

AI calls them *workbenches*. Under the hood they are the same Kubernetes resource, but RHOAI adds additional metadata and dashboard integration.

3.3.1 The KubeFlow Notebook CRD

The KubeFlow Notebook Controller defines a Custom Resource Definition called Notebook in the API group `kubeflow.org/v1` (34). A Notebook resource is structurally similar to a Kubernetes `StatefulSet`: it contains a pod template that specifies the container image, resource requests and limits, volume mounts, and environment variables. When a Notebook resource is created, the Notebook Controller creates the corresponding `StatefulSet`, which then creates a pod running the specified container image — typically a JupyterLab server.

The Notebook Controller also handles lifecycle operations. Setting the annotation `kubeflow-resource-stopped` to a timestamp causes the controller to scale the `StatefulSet` to zero replicas, effectively stopping the notebook without deleting it. Removing the annotation restarts it. This mechanism is how the RHOAI dashboard starts and stops workbenches.

3.3.2 RHOAI Workbenches

In Red Hat OpenShift AI, a *workbench* is a KubeFlow Notebook resource enriched with additional annotations and integrated into the RHOAI dashboard (35). The RHOAI dashboard is the primary way users interact with workbenches today. When a user creates a workbench through the dashboard, it constructs a Notebook manifest with RHOAI-specific annotations such as:

- `opendatahub.io/image-display-name` — the human-readable name of the selected notebook image,
- `opendatahub.io/hardware-profile-name` — the hardware profile governing CPU, memory, and GPU limits,
- `opendatahub.io/username` — the owner of the workbench,
- `notebooks.opendatahub.io/last-image-selection` — a reference to the selected `ImageStream` tag for upgrade tracking.

The dashboard then submits this manifest to the Kubernetes API, where the Notebook Controller picks it up and creates the running

environment. Each workbench is backed by a `PersistentVolumeClaim` that stores the user's data, and is exposed to the user via an OpenShift Route with OAuth authentication.

3.3.3 Notebook Images

RHOAI ships a curated set of notebook images, each used for a specific workflow. Examples include images with pre-installed data-science libraries (NumPy, pandas, scikit-learn), PyTorch or TensorFlow for deep learning, and minimal images for lightweight experimentation and testing. Each image is represented as an OpenShift `ImageStream` (see Section 3.1.2) with one or more version tags. Custom images — for example, an image with a development version of a library not yet available in the default catalogue — can be registered through the RHOAI dashboard or by creating an `ImageStream` directly.

3.3.4 The Current Management Workflow

Today, managing workbenches requires either navigating the RHOAI web dashboard or directly manipulating Kubernetes resources through `oc` or `kubectl`. Both approaches have limitations. The dashboard is user-friendly but manual: creating, stopping, or reconfiguring multiple workbenches is time-consuming. The command-line approach is scriptable but requires deep knowledge of the underlying CRDs, annotations, and resource relationships described in this chapter.

4 Analysis and Design

The previous chapters introduced the Model Context Protocol as a standardised way of connecting AI agents to external tools (Chapter 2) and described Red Hat OpenShift AI workbenches together with the Kubernetes resources that underlie them (Chapter 3). This chapter bridges the gap between that background knowledge and the implementation itself. It begins by stating the motivation for the project and the concrete goals it aims to achieve, then derives a set of functional and non-functional requirements, justifies the key technology choices, and presents the architecture of the server.

4.1 Motivation and Goals

The introduction outlined the core problem: workbench management in RHOAI is either manual (dashboard) or requires deep Kubernetes expertise (command line). Section 3.3.4 described these workflows in detail. This thesis proposes an MCP-based solution that lets users interact with the cluster through natural language, delegating the underlying complexity to an AI agent.

As a concrete example, a data scientist could request: “create three workbenches with an image containing PyTorch and pandas, each having 4 CPUs and 16 GiB of memory.” The agent interprets this request, selects the appropriate MCP tool, constructs the parameters from the KubeFlow Notebook CRD and RHOAI annotations (Section 3.3.2), and executes the operation — all without the user needing to know how workbenches are represented in Kubernetes.

It is worth noting that the current implementation targets the existing KubeFlow-based workbench architecture described in Chapter 3 and is intended as a proof of concept. Notebooks 2.0 (8), a redesigned notebook subsystem, is expected to be released and integrated into RHOAI later this year. The implications of this transition and the changes it will require in the MCP server are discussed in Section 6.1.

With this motivation in mind, the thesis pursues the following goals:

1. **Full workbench lifecycle coverage.** Provide MCP tools for the complete set of workbench management operations: listing, creating, updating, and deleting workbenches, notebook images, hardware profiles, persistent volume claims, and namespaces.
2. **Resource consumption monitoring.** Provide tools to query resource consumption (CPU, memory, disk, and GPU) at multiple granularities — per workbench, per namespace, per user, and per cluster — giving administrators and data scientists visibility into how compute resources are being used.
3. **Safety by default.** Operate in a read-only mode by default, exposing only listing and querying tools. Write operations — creation, modification, and deletion — are gated behind an explicit opt-in mechanism so that users can explore the server’s capabilities without risk of accidental modifications to the cluster.
4. **Quality assurance.** Validate correctness through unit tests and automated evaluation of end-to-end agent behaviour, measuring tool selection accuracy and parameter extraction correctness. This also serves the proof-of-concept purpose by establishing a methodology for evaluating MCP server implementations.
5. **Exploration of all MCP primitives.** Since the implementation serves as a proof of concept, exercise the full breadth of the MCP standard — not only tools, but also resources and prompts. Specifically, expose MCP resources (an image catalogue, available hardware profiles, and cluster namespaces) and an MCP prompt that orchestrates a guided workbench creation workflow, demonstrating how the three server primitives can work together in practice.

4.2 Requirements

This section translates the goals from Section 4.1 into concrete requirements. Functional requirements describe *what* the system must do; non-functional requirements describe *how* it must behave.

4.2.1 Functional Requirements

1. **FR-1: Workbench management.** The server must provide tools to list, create, update, and delete workbenches. Listing must support both a single-namespace and a cross-namespace mode. Each workbench entry must include the workbench name, notebook image, hardware profile, and attached persistent volume claim.
2. **FR-2: Workbench lifecycle control.** The server must provide a tool to start and stop a workbench without deleting it, using the `kubeflow-resource-stopped` annotation mechanism described in Section 3.3.1.
3. **FR-3: Notebook image management.** The server must provide tools to list all available notebook images (including version tags, Python dependencies, and bundled software), create custom images by registering new ImageStreams, update existing images, and delete them.
4. **FR-4: Hardware profile management.** The server must provide tools to list, create, update, and delete hardware profiles, each specifying resource identifiers (CPU, memory, and optionally GPU) with default, minimum, and maximum values.
5. **FR-5: Storage management.** The server must provide tools to list, create, resize, and delete persistent volume claims (PVCs). Resizing supports expansion only, as Kubernetes does not support shrinking.
6. **FR-6: Namespace and pod listing.** The server must provide tools to list all namespaces in the cluster and to list pods in a given namespace.
7. **FR-7: Resource consumption monitoring.** The server must provide tools to query resource consumption (CPU, memory, disk, and GPU) at four granularities: per workbench, per namespace, per user, and per cluster.

8. **FR-8: MCP resources.** The server must expose at least one MCP resource to demonstrate the resource primitive and its usage in a real-world scenario.
9. **FR-9: MCP prompt.** The server must expose at least one MCP prompt to demonstrate the prompt primitive and its usage in a real-world scenario.

4.2.2 Non-Functional Requirements

1. **NFR-1: Safety.** The server must start in read-only mode by default. Write operations (create, update, delete) must only be registered when explicitly enabled via the `-mode write` command-line flag or the `MCP_RHOAI_MODE=write` environment variable.
2. **NFR-2: Host independence.** The server must communicate exclusively through the MCP standard and must not contain any host-specific code. It must be usable from any MCP-compatible host.
3. **NFR-3: Local cluster authentication.** The server does not manage its own authentication. The user must be logged in to an OpenShift cluster locally (via `oc login`) before starting the server. All Kubernetes API calls are executed using the credentials stored in the user's kubeconfig file, and therefore run with the permissions of the logged-in user.
4. **NFR-4: Code quality.** The codebase must pass static analysis and linting checks, and follow clear package separation.
5. **NFR-5: Unit testing.** All tools must have unit tests covering both success and error paths.
6. **NFR-6: Evaluation.** End-to-end agent behaviour must be evaluated using an automated framework that measures tool selection accuracy, parameter extraction correctness, and execution success rate.

4.3 Technology Choices

This section justifies and describes the key technology used for the implementation.

Go. The server is written in Go (36). Go was chosen for two reasons. First, it is the dominant language in the Kubernetes ecosystem — Kubernetes itself, OpenShift, and the Kubeflow Notebook Controller are all written in Go, and the official Kubernetes client library (`client-go`) is a Go module. Using Go means the server can import `client-go` directly without a foreign-function interface or an HTTP wrapper. A Python or TypeScript implementation would need to communicate with the cluster through a generated REST client or a wrapper library rather than using the same native client that Kubernetes itself uses. Second, Go compiles to a single static binary with no runtime dependencies, which simplifies distribution — users only need to place the binary on their `PATH` and point their MCP host configuration at it. By contrast, a Python-based server would require the user to have a compatible Python interpreter and manage a virtual environment with the correct dependencies installed.

MCP Go SDK. Given the choice of Go as the implementation language, the server uses the official MCP Go SDK (37) (version 1.1.0) for all protocol-level concerns. The alternative would be implementing the MCP protocol from scratch, which would require manually handling JSON-RPC message parsing, capability negotiation, and JSON Schema generation for tool inputs — a significant effort that would duplicate work already done in the official SDK. Using the official SDK also ensures protocol compliance as the specification evolves. The SDK handles creating the MCP server instance, registering tools with their JSON Schema input definitions (derived automatically from Go struct tags), registering resources and prompts, and running the transport loop. Its typed tool registration function (`mcp.AddTool`) accepts a Go struct as the input type and automatically generates the JSON Schema that is advertised to the host, eliminating manual schema maintenance and reducing the risk of inconsistencies between the declared schema and the actual implementation.

golangci-lint. Static analysis is performed by `golangci-lint` (38), a tool that runs multiple Go linters and aggregates their results into a single invocation. The alternative would be running each linter separately (`go vet`, `errcheck`, `staticcheck`, etc.), which complicates build configuration and slows down the feedback loop. `golangci-lint` is the de facto standard in the Go ecosystem: it runs linters in parallel for speed, provides a unified configuration format, and is widely adopted by open-source Go projects. It is integrated into the build process through the Makefile: `make build` runs `golangci-lint run` before compiling the binary, ensuring that code quality checks pass on every build.

stdio transport. The server uses the stdio transport (Section 2.3.5) exclusively, initialised in the entry point as:

```
server.Run(context.Background(), &mcp.  
    StdioTransport{})
```

This choice reflects the intended deployment model: the MCP server runs as a local process on the same machine as the AI host, typically the developer's workstation. Although the server communicates with a remote Kubernetes cluster (via the user's `kubeconfig` credentials), the MCP protocol itself operates locally between the host application and the server. An HTTP/SSE transport would make sense if the MCP server were deployed as a shared network service accessed by multiple users, but this would introduce additional complexity — TLS certificates, authentication tokens, and service deployment — without clear benefit for the single-user, proof-of-concept scope of this implementation. The stdio transport requires no network configuration, is inherently secure (no exposed ports), and integrates seamlessly with how MCP hosts such as Cursor and Claude Desktop manage local tool servers. If multi-user or remote deployment is needed in the future, switching to the HTTP/SSE or Streamable HTTP transport would be straightforward at the code level (only the transport initialisation changes), but would also require setting up TLS certificates, an authentication layer, and deploying the server as a network service.

Promptfoo. End-to-end evaluation uses the Promptfoo framework (39), an open-source tool for testing LLM applications. Alternatives such as DeepEval (40) were considered — DeepEval offers over fifty evaluation metrics, including a dedicated tool-correctness metric, but that breadth introduces complexity that is unnecessary when the only goal is to evaluate tool-calling behaviour. Promptfoo is more focused: its entire configuration fits into a single YAML file, it requires no custom evaluation code, and it directly supports the workflow needed here — sending natural-language prompts to an LLM, capturing the tool calls the model emits, and checking them against expected outcomes. It measures tool selection accuracy and parameter extraction correctness. It also supports multiple LLM providers (OpenAI, Anthropic, Google), making it straightforward to compare tool-calling quality across models.

4.4 Architecture

This section describes the internal structure of the MCP server: how the codebase is organised, how the two operation modes are implemented, how the server communicates with the OpenShift cluster, and how the project is built.

4.4.1 Project Structure

The project root contains two key files: `main.go`, which is the entry point that creates the MCP server instance, registers tools, resources, and prompts, and starts the stdio transport loop; and a `Makefile` that drives the build pipeline described in Section 4.4.4. The remaining code is organised into four packages:

- `core` — shared type definitions (input/output structs for every tool), Kubernetes `GroupVersionResource` constants, and cluster authentication helpers.
- `tools` — the implementation of every MCP tool, plus a registry module that registers tools with the MCP server. The registry exposes two functions: `RegisterReadOnlyTools`, which registers the listing and monitoring tools that are always available, and `RegisterWriteTools`, which additionally registers the

create, update, and delete tools. The entry point inspects the `MCP_RHOAI_MODE` environment variable and calls one or both functions accordingly.

- **resources** — the implementations of the three MCP resources: the Image Catalog, Hardware Profiles, and Namespaces. A registry module registers all resources with the MCP server.
- **prompts** — the implementation of the Create Workbench MCP prompt, together with a registry module that registers it with the MCP server.

4.4.2 Two-Mode Design

The server's safety model is implemented at the registration level rather than at the execution level. When the server starts, it determines the operation mode from two sources: the `MCP_RHOAI_MODE` environment variable and the `-mode` command-line flag. The flag, if provided, takes precedence over the environment variable, allowing users to override the mode without modifying their shell environment. If the resolved mode is `write`, the server calls `RegisterWriteTools` followed by `RegisterReadOnlyTools`, making all tools available. For any other value (or when neither the flag nor the variable is set), only `RegisterReadOnlyTools` is called. Because tools that are not registered are never advertised to the host during capability negotiation, the LLM cannot invoke them — it does not know they exist. This approach is simpler and more robust than runtime permission checks, since there is no code path that could accidentally bypass a guard.

4.4.3 Kubernetes Client Strategy

The server uses two types of Kubernetes clients from the `client-go` library (41). The **typed client** (`kubernetes.Clientset`) is used for operations on built-in resources such as pods and namespaces, where the Go type system provides compile-time safety. The **dynamic client** (`dynamic.DynamicClient`) is used for custom resources — Kubeflow Notebooks, OpenShift ImageStreams, and hardware profiles — where no generated Go types are available and the server must work with `unstructured.Unstructured` objects. Both clients are constructed

from the user's `kubeconfig` file, reusing whatever credentials and permissions the user has when they run `oc login`.

To support unit testing, both clients are exposed as package-level function variables (`GetClientSet` and `GetDynamicClient`) that can be replaced with mock implementations in test code, satisfying NFR-5.

4.4.4 Build Pipeline

The project uses a Makefile to integrate all quality checks into a single build command. Running `make build` executes three stages in sequence: first `golangci-lint run` to verify that the code passes all configured linters, then `go test` to run the unit test suite, and finally `go build` to compile the binary. If any stage fails, the build stops immediately. This ensures that every produced binary has passed both linting and tests. Evaluation is run separately via `make eval`, which invokes Promptfoo against the compiled server.

5 Implementation

The previous chapter defined the requirements, technology choices, and architecture of the MCP server. This chapter describes how the server was implemented. It walks through the specific MCP tools, resources, and the prompt that were implemented. The chapter closes with a discussion of security considerations, code quality practices, and the evaluation methodology.

5.1 MCP Tools

The server exposes 20 MCP tools covering five resource domains. Each subsection below opens with a table listing its tools and then discusses the design decisions and non-trivial implementation details. In all tables, tools marked (*write*) are only registered when the environment variable `MCP_RHODAI_MODE=write` is set; all other tools are always available. Notably, several listing operations that require no parameters (such as listing images, hardware profiles, and namespaces) are exposed as MCP resources rather than tools, providing a cleaner interface for clients (Section 5.2).

Each tool is registered with the MCP server using the SDK's `mcp.AddTool` function, which accepts a `mcp.Tool` descriptor (name and description) and a Go function that implements the tool's logic. Every tool function follows the same high-level pattern: obtain a Kubernetes client, perform the required API call, transform the result into a typed output struct, and return it. Errors are propagated back to the host as standard MCP error responses.

5.1.1 Workbench Management

Table 5.1 lists the five workbench management tools. Workbench management is the most complex part of the server, comprising tools that operate on Kubeflow Notebook resources (API group `kubeflow.org/v1`) through the dynamic Kubernetes client.

Listing. The `list_workbenches` tool is the foundation on which several other tools are built. For each Notebook resource in the

Table 5.1: Workbench management tools.

Tool	Description
<code>list_workbenches</code>	List workbenches (optionally filtered by namespace)
<code>create_workbench</code>	Create a new workbench (<i>write</i>)
<code>update_workbench</code>	Update image, hardware profile, or PVC (<i>write</i>)
<code>delete_workbench</code>	Delete a workbench (<i>write</i>)
<code>change_workbench_status</code>	Start or stop a workbench (<i>write</i>)

specified namespace, the tool reads RHOAI-specific annotations to extract the image display name, image tag, hardware profile name, and owner. Running status is determined by the presence of the `kubeflow-resource-stopped` annotation: if set, the workbench is stopped; otherwise, it is running. For running workbenches, it additionally queries the typed client for pods labelled `notebook-name=<workbenchName>` to obtain uptime from the pod’s start time. Disk usage is read from the PVC’s requested storage capacity. The result is a list of `WorkbenchInfo` structs containing the workbench name, user, status, image, hardware profile, PVC name, namespace, uptime, and resource consumption (CPU, memory, disk, GPU). When the namespace parameter is empty or not provided, the tool uses an empty namespace string in the Kubernetes API call, which returns results across all namespaces, giving administrators a cluster-wide view.

Creation. The `create_workbench` tool is the most complex in the server. It must resolve the user-specified image display name and tag to a concrete container image URL by looking up the corresponding `ImageStream`, optionally create a PVC (defaulting to the workbench name and 10 GiB if none is provided), and fall back to the default hardware profile (Section 5.2) when no profile is specified. The manifest is constructed as an unstructured. Unstructured object with the same annotations the RHOAI dashboard would set — `opendatahub.io/image-display-name` and `notebooks.opendatahub.io/last-image-selection`, among others.

The container spec includes JupyterLab server arguments, two volume mounts (the user’s PVC and a shared-memory `emptyDir`), and resource requests and limits derived from the hardware profile. After submission, the Notebook Controller creates the running environment. This design ensures that workbenches created through the MCP server are indistinguishable from those created through the dashboard.

Updates and lifecycle control. The `update_workbench` tool supports three independent update dimensions — the notebook image, the hardware profile, and the attached PVC — each applied only when the corresponding input fields are non-empty, allowing partial updates. The `change_workbench_status` tool starts or stops a workbench by manipulating the `kubeflow-resource-stopped` annotation: setting it to the current timestamp stops the workbench; removing it (via a `nil` merge patch) restarts it. Before making any change, the tool checks the current state and returns early if the workbench is already in the requested state, avoiding redundant Kubernetes API calls. The `delete_workbench` tool issues a single `Delete` call; the Notebook Controller handles cleanup of the associated `StatefulSet` and `pod`, but the PVC is intentionally preserved so that the user’s data is not lost.

5.1.2 Hardware Profiles

Table 5.2: Hardware profile tools.

Tool	Description
<code>create_hardware_profile</code>	Create a new profile (<i>write</i>)
<code>update_hardware_profile</code>	Merge-update profile resources (<i>write</i>)
<code>delete_hardware_profile</code>	Delete a profile (<i>write</i>)

Table 5.2 lists the three tools that manage hardware profiles (API group `infrastructure.opendatahub.io/v1`). Listing available profiles is handled by the Hardware Profiles MCP resource (Section 5.2.2), which returns each profile’s resource identifiers with their default, minimum, and maximum values. Creation and deletion are straightforward operations on `unstructured.Unstructured` objects.

The more interesting case is `update_hardware_profile`, which implements a *merge* strategy: it fetches the existing profile, iterates over the provided resource entries, and overwrites any entry whose resource identifier already exists while appending new ones. This means the agent can add a GPU to an existing profile without having to re-specify the CPU and memory entries, which makes the tool more natural to use through conversational interaction.

5.1.3 Notebook Images

Table 5.3: Notebook image tools.

Tool	Description
<code>create_custom_image</code>	Register a custom image (<i>write</i>)
<code>update_image</code>	Update display name or description (<i>write</i>)
<code>delete_image</code>	Delete with safety checks (<i>write</i>)

Table 5.3 lists the three notebook image tools. Notebook images are represented as OpenShift ImageStreams (API group `image.openshift.io/v1`) labelled with `opendatahub.io/notebook-image=true`. Listing available images is handled by the Image Catalog MCP resource (Section 5.2.1), which formats each image’s display name, container image URL, and version tags with their Python dependencies into a structured JSON response. The `create_custom_image` tool registers a new ImageStream with a single latest tag, setting the labels and annotations required for dashboard visibility. The `update_image` tool modifies the display name or description annotations.

The most notable tool in this group is `delete_image`, which enforces two safety checks before proceeding. First, it verifies that the image is not a default RHOAI image by inspecting the `internal.config.kubernetes.io/previousNamespaces` annotation — default images are managed by the platform and should not be removed by users. Second, it scans all workbenches across all namespaces and compares the `opendatahub.io/image-display-name` annotation to ensure the image is not currently in use. If either check fails, the deletion is rejected with an explanatory error message.

These checks prevent accidental removal of images that would leave running workbenches in a broken state.

5.1.4 Storage and Infrastructure

Table 5.4: Storage and infrastructure tools.

Tool	Description
<code>list_pvc</code> s	List PVCs in a namespace
<code>create_pvc</code>	Create a new PVC (<i>write</i>)
<code>update_pvc</code>	Resize a PVC (expansion only) (<i>write</i>)
<code>delete_pvc</code>	Delete a PVC (<i>write</i>)
<code>list_pods</code>	List pods in a namespace (<i>write</i>)

Table 5.4 lists the storage and infrastructure tools. The `list_pvc`s tool lists all PVCs in a namespace; listing cluster namespaces is handled by the Namespaces MCP resource (Section 5.2.3). The `list_pods` tool (registered only in write mode) provides basic cluster inspection, useful for debugging workbench issues such as `CrashLoopBackOff` states.

The `create_pvc` tool creates a PVC with the specified name and size. The `update_pvc` tool handles PVC resizing, comparing the requested size with the current size using the `resource.Quantity` type from the Kubernetes API toolkit. If the new size is smaller, the update is rejected with an error, since Kubernetes does not support shrinking PVCs. The `delete_pvc` tool removes a PVC by name.

5.1.5 Resource Consumption

Table 5.5 lists the four read-only tools that provide resource consumption monitoring at different granularities: per workbench, per namespace, per user, and cluster-wide. All four report the same four resource types — CPU, memory, disk, and GPU — and are always available regardless of the server mode.

The values these tools report are based on Kubernetes resource *requests* declared in the workbench’s container spec and the PVC’s requested storage capacity, not on live metrics. This means they reflect

Table 5.5: Resource consumption monitoring tools (all read-only).

Tool	Description
<code>list_resource_consumption_per_workbench</code>	Per-workbench consumption
<code>list_resource_consumption_per_namespace</code>	Aggregated per namespace
<code>list_resource_consumption_per_user</code>	Aggregated per user
<code>list_resource_consumption_per_cluster</code>	Aggregated cluster-wide

what the workbench has *reserved*, which is the relevant quantity for capacity planning.

The per-workbench tool simply filters the output of the workbench listing logic by name. The three aggregating tools (per-namespace, per-user, and cluster-wide) all use a shared `sumResourceValues` helper that parses Kubernetes quantity strings (e.g. 4Gi, 500m), accumulates the numeric parts, and preserves the unit from the first non-zero value. The per-user tool filters by the `opendatahub.io/username` annotation across all namespaces, enabling administrators to answer questions such as “how many resources has user X reserved across the cluster?”

5.2 MCP Resources and Prompts

Beyond tools, the server exposes three MCP resources and one MCP prompt. Together with the tools described above, these demonstrate all three MCP server primitives in a real-world scenario. Notably, listing operations that require no parameters — images, hardware profiles, and namespaces — are exposed as resources rather than tools. This design follows the MCP specification’s guidance that resources are appropriate for data that the client can read without needing to provide input parameters.

5.2.1 Image Catalog Resource

The Image Catalog is registered with the URI `resource://mcp-server-rhoai/images` and the MIME type

application/json. When read, it calls the `GetImages` function, which queries all `ImageStreams` in the default RHOAI namespace that carry the `opendatahub.io/notebook-image=true` label. For each image, it extracts the display name from annotations, the container image URL from the `ImageStream's status.dockerImageRepository` field, and all version tags with their Python dependency and software annotations. The result is serialised as a JSON array and returned to the host. An agent can read this resource before creating a workbench to discover which images are available and select a suitable one.

5.2.2 Hardware Profiles Resource

The `Hardware Profiles` resource is registered with the URI `resource://mcp-server-rhoai/hardware-profiles` and the MIME type `application/json`. When read, it queries all `HardwareProfile` custom resources in the RHOAI namespace and returns each profile's resource identifiers with their default, minimum, and maximum values. This resource replaces what was previously a listing tool, providing a cleaner interface for clients that simply need to discover available profiles before creating or updating a workbench.

5.2.3 Namespaces Resource

The `Namespaces` resource is registered with the URI `resource://mcp-server-rhoai/namespaces` and the MIME type `application/json`. When read, it queries the Kubernetes API for all namespaces in the cluster and returns them as a JSON array of namespace names. This allows an agent to discover available namespaces before performing operations that require a namespace parameter, such as creating workbenches or listing PVCs.

5.2.4 Create Workbench Prompt

The `Create Workbench` prompt is registered under the name `create-workbench-prompt` with two required arguments: `namespace` and `name`. When invoked, it returns a single user-role message that instructs the agent to consult the `Image Catalog` resource to find a suitable Data Science image (preferring Python 3.12) and then call

the `create_workbench` tool. This prompt demonstrates how the three MCP primitives interact in practice: a prompt guides the agent to first read a resource (the Image Catalog) and then invoke a tool (Create Workbench), orchestrating a multi-step workflow through a single user action.

5.3 Security

Security is addressed at two levels: the two-mode design described in Section 4.4.2, which prevents accidental cluster modifications, and the Kubernetes authentication model.

The server does not implement its own authentication or authorisation. Instead, it relies entirely on the user's existing OpenShift credentials. Both the typed and dynamic Kubernetes clients are constructed from the `kubeconfig` file at `~/.kube/config`, which stores the token obtained via `oc login`. Every API call the server makes therefore runs with the permissions of the logged-in user; if the user does not have permission to, for example, delete workbenches in a certain namespace, the Kubernetes API server rejects the request and the error is propagated back to the agent.

This delegation model has two advantages. First, it avoids introducing a separate credential management system — the server reuses the same authentication flow that platform engineers already use. Second, it ensures that the server cannot escalate privileges beyond what the user already has, since every API call is subject to the same permission checks that the cluster enforces for all users.

The image deletion tool includes additional application-level safety checks. It refuses to delete images that are marked as RHOAI defaults (by inspecting the `internal.config.kubernetes.io/previousNamespaces` annotation) and images that are currently in use by any workbench (by scanning all workbenches across namespaces). These checks prevent accidental removal of images that would break existing workbenches.

5.4 Code Quality

Code quality is maintained through static analysis and unit testing, both integrated into the build pipeline.

5.4.1 Static Analysis

The project uses `golangci-lint` with five enabled linters: `errcheck` (ensures error return values are checked), `govet` (reports suspicious constructs), `ineffassign` (detects ineffectual assignments), `staticcheck` (a suite of advanced static analysis checks), and `unused` (finds unused code). Every build runs `golangci-lint run` as the first stage; if any linter reports an issue, the build fails immediately.

5.4.2 Unit Testing

Unit tests validate tool behaviour against a simulated OpenShift cluster without requiring access to a live environment. This is made possible by the injectable client design described in Section 4.4.3: the `GetClientSet` and `GetDynamicClient` function variables in the `tools` package are replaced with fake implementations in test code. The common setup logic is centralised in a shared `test_helpers_test.go` file that exposes `setupFakeClient` and `setupFakeClientSet` helpers; each helper injects a `client-go` fake into the package-level variable and restores the original on cleanup. Some tools that depend on other tools internally (for example, the resource consumption tools call the workbench listing logic) use additional function-level mocks such as `listWorkbenchesFn` that are replaced in tests with simple functions returning predefined data. The test suite comprises eight test files covering every tool domain:

- **Workbenches** — tests for listing (stopped and running workbenches), creation (including automatic PVC creation), updates, deletion, and status changes. Pure helper functions such as image tag parsing and resource requirement construction are tested separately.
- **Hardware profiles** — listing, creation, update, deletion, and parsing of profile resource identifiers.

- **Notebook images** — creation, update, and deletion of custom images, including verification that default images and images in use by workbenches cannot be deleted. The `ListImages` tool and the underlying `GetImages` function (which backs the Image Catalog resource) are tested with fake `ImageStream` objects.
- **Storage** — PVC creation, deletion, resize (verifying that shrinking is rejected), and listing.
- **Namespaces and pods** — listing with typed client fakes.
- **Resource consumption** — the aggregation helpers (`parseResourceValue`, `sumResourceValues`) are tested directly, and all four granularity tools are tested with predefined workbench data.

5.5 Evaluation

Unit tests verify that each tool behaves correctly in isolation, but they cannot answer the question that matters most: will an AI agent actually use these tools successfully? The value of an MCP server depends entirely on whether language models can interpret natural-language requests, select appropriate tools, and extract correct parameters from conversational input. This makes end-to-end evaluation essential.

Evaluating LLM-based systems presents a fundamental challenge: language models are inherently non-deterministic. The same prompt may produce different tool calls across runs due to sampling randomness, and minor prompt variations can lead to entirely different tool selections. A robust evaluation must therefore test a representative set of scenarios and accept that perfect reproducibility is not achievable. The goal is to verify that the model *generally* selects the right tools and extracts the right parameters, not that it does so identically every time.

End-to-end evaluation uses the `Promptfoo` framework (39) to measure how well an LLM agent uses the MCP server’s tools in response to natural-language requests. `Promptfoo` was chosen for its simplicity — the entire evaluation is configured in a single YAML file (`promptfoo.yaml`) with no custom evaluation code required, and it can be run with `make eval`.

5.5.1 Evaluation Setup

Promptfoo connects to the compiled MCP server binary as an MCP provider, starting the server with the `-mode write` flag so that all tools are available. The active provider in the configuration is Google's Gemini 2.5 Pro; configurations for OpenAI GPT-4o, GPT-4o-mini, and Anthropic Claude 3.5 Sonnet are included as comments, making it straightforward to compare tool-calling quality across models.

Each test case consists of a natural-language prompt (the `prompt` variable) and one or more assertions. The assertions use Promptfoo's `tool-call-f1` metric, which checks whether the set of tools the model actually called matches the expected set. This metric captures both *precision* (the model did not call unnecessary tools) and *recall* (the model called all required tools), combining them into a single F1 score.

5.5.2 Test Cases

The evaluation suite includes 22 test cases organised into seven categories, providing comprehensive coverage of all tool domains:

- **Workbench listing** (2 tests) — listing workbenches cluster-wide and within a specific namespace, verifying that the model selects the appropriate tool variant and extracts namespace parameters.
- **Workbench CRUD** (5 tests) — creating, updating, and deleting workbenches, plus starting and stopping them via the status change tool.
- **Pods** (1 test) — listing pods within a specific namespace.
- **Notebook images** (3 tests) — creating custom images with name and location, updating image metadata, and deleting images.
- **Hardware profiles** (3 tests) — creating, updating, and deleting hardware profiles with resource specifications.
- **Storage** (3 tests) — listing PVCs in a namespace, creating a PVC with a specified size, and resizing an existing PVC.
- **Resource consumption** (5 tests) — querying consumption at all four granularities (per-workbench, per-namespace, per-user, and cluster-wide), with multiple prompt variations to test robustness.

Promptfoo evaluates only tool-calling behaviour; it does not support testing MCP resources or prompts. Consequently, the listing op-

erations that are exposed as resources (available notebook images, hardware profiles, and namespaces) are not covered by this automated evaluation. Their correctness is verified through manual testing and unit tests instead.

Each test case verifies both tool selection (via the `tool-call-f1` metric) and parameter extraction (via contains assertions that check for expected argument values in the tool call). A 2-second delay is configured between test cases to avoid API rate limiting.

5.5.3 Running the Evaluation

The evaluation requires an API key for the configured LLM provider. The default configuration uses Google Gemini, so users must set the `GOOGLE_API_KEY` environment variable before running. The evaluation is then invoked via `make eval`, which runs `npx promptfoo@latest eval -c promptfoo.yaml`. Results are written to `promptfoo-results.json` and can also be explored in a web-based dashboard via `make eval-view`. The YAML-based configuration makes it straightforward to add new test cases, switch providers, or adjust assertion types as the tool set evolves.

5.5.4 Results and Learnings

The evaluation was run using Google Gemini 2.5 Pro as the LLM provider. The evaluation was executed multiple times to verify consistency, and every run achieved a 100% pass rate: all 22 test cases passed, with 52 individual assertions verified successfully. This consistent result across repeated runs demonstrates that the tool names, descriptions, and parameter schemas are sufficiently clear for a state-of-the-art language model to reliably select the correct tool and extract the required parameters from natural-language requests.

Running the evaluation iteratively during development proved valuable for improving tool descriptions. Early runs revealed cases where the model selected incorrect tools or failed to extract parameters, which pointed to ambiguities in the tool definitions. Several refinements emerged from this process:

- Optional fields must be explicitly marked with `omitempty` in the Go struct tags, and the JSON schema descriptions should state the default value that will be used when the field is omitted.
- Enumerated values should be listed explicitly in the description — for example, the `change_workbench_status` tool describes its status parameter as “0 for running, 1 for stopped” rather than relying on the model to infer the mapping.
- Testing with different models (Gemini 2.5 Pro, Gemini 2.5 Flash, Claude 3.5 Sonnet) exposed robustness issues: simpler models are stricter about parameter formats, making them useful for catching under-specified descriptions that more capable models might tolerate.

These learnings apply broadly to MCP server development: the evaluation framework not only validates correctness but also serves as a feedback loop for improving tool usability across different language models.

6 Conclusion

This thesis presented the design and implementation of an MCP server for Red Hat OpenShift AI workbenches. The server is written in Go using the official MCP Go SDK and communicates over the stdio transport, making it usable from any MCP-compatible host without host-specific code. The complete source code is released as open source under the Apache License 2.0 and is available at <https://github.com/amaly/mcp-server-rhoai> (42).

The five goals stated in Section 4.1 have been met. The server provides **full workbench lifecycle coverage** through 20 tools covering five resource domains: workbenches, notebook images, hardware profiles, persistent storage, and namespaces. **Resource consumption monitoring** is available at four granularities — per workbench, per namespace, per user, and cluster-wide — giving both data scientists and administrators visibility into how compute resources are allocated. **Safety by default** is enforced at the registration level: the server starts in read-only mode and only exposes write tools when explicitly enabled via the `-mode` command-line flag or an environment variable, so that unregistered tools are never advertised to the host. **Quality assurance** is covered by unit tests that use centralised fake-client helpers to validate every tool domain against a simulated cluster, and by an automated Promptfoo evaluation comprising 22 test cases that achieved a 100% pass rate with Gemini 2.5 Pro — verifying both tool selection and parameter extraction. The iterative evaluation process also helped refine tool descriptions for better cross-model compatibility. Finally, the server **exercises all three MCP primitives** — tools, three resources (Image Catalog, Hardware Profiles, and Namespaces), and a Create Workbench prompt — demonstrating how they can all work together in a guided workflow.

Because RHOAI is transitioning from the current KubeFlow-based workbench architecture to Notebooks 2.0 — a redesigned notebook subsystem described in the following section — the implementation presented in this thesis is best understood as a proof of concept. It demonstrates that managing OpenShift AI workbenches through natural language via MCP is both feasible and practical, and establishes the architectural patterns, tool design, and evaluation methodology that a

future production-ready server can build upon. Once Notebooks 2.0 is integrated into RHOAI, the server will need to be restructured to target the new API surface, but the core design decisions — the two-mode safety model, the package separation, and the MCP-level infrastructure — are independent of the underlying CRD and can be carried forward.

6.1 Future Work: Notebooks 2.0

Kubeflow Notebooks 2.0, officially called *Kubeflow Workspaces* (8), is a complete redesign of the notebook subsystem that is expected to be integrated into RHOAI later this year. The redesign replaces the single Notebook CRD (kubeflow.org/v1) with two new custom resources under the API group kubeflow.org/v1beta1 (43): a namespaced Workspace resource that end users create, and a cluster-scoped WorkspaceKind resource that administrators define as a template. When creating a Workspace, users select a WorkspaceKind (e.g. “jupyterlab”, “vs-code”), an approved image configuration, and a pod configuration (e.g. “small_cpu”, “big_gpu”) from options defined by the administrator in the WorkspaceKind; they no longer need to specify low-level Kubernetes details such as container image URLs, resource requests, or volume mount paths. Metadata that the current architecture stores as RHOAI-specific annotations — image display names, hardware profile references, workbench ownership, and the stopped/running state — moves into structured spec and status fields. The status model is also richer: instead of the current binary running/stopped distinction, a Workspace reports one of six states (Running, Paused, Pending, Terminating, Error, Unknown). Additional new features include built-in activity culling with configurable probes, deferred updates that let administrators redirect old image or pod configurations to new ones without breaking existing workspaces, and explicit separation of home and data volumes in the Workspace spec.

Adapting the MCP server to Notebooks 2.0 would require changes in three main areas. First, the workbench tools would need to target the new Workspace CRD instead of the current Notebook CRD — reading structured spec fields rather than parsing annotations, and cre-

ating workbenches by referencing `WorkspaceKind` option identifiers rather than constructing full Kubernetes manifests. Second, image and hardware profile management would change most significantly, because approved images and compute profiles are no longer separate resources (`ImageStreams` and `HardwareProfiles`) but entries defined inside `WorkspaceKind` objects. Third, the MCP resources and the Create Workbench prompt would need to query `WorkspaceKind` resources as their data source. On the other hand, the tool registry, the two-mode safety design, the MCP-level infrastructure, the evaluation framework, and the overall project structure can all remain unchanged. Notebooks 2.0 could also enable entirely new MCP tools — for example, listing available `WorkspaceKind` definitions or querying workspace activity and idle status.

Attachments

The electronic attachment accompanying this thesis contains the following:

mcp-server-rhoai/ The complete source code of the MCP server implementation described in Chapter 5. The directory structure mirrors the project layout described in Section 4.4.1, including:

- `main.go` — the entry point
- `core/` — shared types and Kubernetes client helpers
- `tools/` — all 20 MCP tool implementations
- `resources/` — MCP resource implementations
- `prompts/` — MCP prompt implementation
- `*_test.go` — unit test files
- `Makefile` — build and evaluation targets
- `promptfoo.yaml` — evaluation configuration

License. The source code is released under the Apache License 2.0, a permissive open-source license that allows commercial use, modification, and distribution.

Upstream repository. The canonical version of the source code is maintained at <https://github.com/amaly/mcp-server-rhoai>. The attached version corresponds to the state of the repository at the time of thesis submission.

Bibliography

1. BROWN, Tom; MANN, Benjamin; RYDER, Nick; SUBBIAH, Melanie; KAPLAN, Jared D; DHARIWAL, Prafulla; NEELAKANTAN, Arvind; SHYAM, Pranav; SASTRY, Girish; ASKELL, Amanda, et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. 2020, vol. 33, pp. 1877–1901. Available also from: <https://arxiv.org/abs/2005.14165>.
2. YAO, Shunyu; ZHAO, Jeffrey; YU, Dian; DU, Nan; SHAFRAN, Izhak; NARASIMHAN, Karthik; CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. In: *International Conference on Learning Representations*. 2023. Available also from: <https://arxiv.org/abs/2210.03629>.
3. ANTHROPIC. *Introducing the Model Context Protocol* [online]. 2024-11-25. [visited on 2026-03-10]. Available from: <https://www.anthropic.com/news/model-context-protocol>.
4. RED HAT. *Red Hat OpenShift AI Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/.
5. RED HAT. *Red Hat OpenShift* [online]. 2024. [visited on 2026-05-03]. Available from: <https://www.redhat.com/en/technologies/cloud-computing/openshift>.
6. THE KUBERNETES AUTHORS. *Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/home/>.
7. PROJECT JUPYTER. *JupyterLab Documentation* [online]. 2024. [visited on 2026-05-03]. Available from: <https://jupyterlab.readthedocs.io/>.
8. WICKS, Mathew. *Kubeflow Notebooks 2.0 (Kubeflow Workspaces)* — *GitHub Issue #85* [online]. 2023. [visited on 2026-04-25]. Available from: <https://github.com/kubeflow/notebooks/issues/85>.
9. VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N; KAISER, Łukasz; POLOSUKHIN, Illia. Attention Is All You Need. *Advances in Neural Information Processing Systems*. 2017, vol. 30. Available also from: <https://arxiv.org/abs/1706.03762>.

10. OUYANG, Long; WU, Jeffrey; JIANG, Xu; ALMEIDA, Diogo; WAINWRIGHT, Carroll; MISHKIN, Pamela; ZHANG, Chong; AGARWAL, Sandhini; SLAMA, Katarina; RAY, Alex, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*. 2022, vol. 35, pp. 27730–27744. Available also from: <https://arxiv.org/abs/2203.02155>.
11. OPENAI. *ChatGPT* [online]. 2022. [visited on 2026-05-03]. Available from: <https://openai.com/chatgpt>.
12. ANTHROPIC. *Claude* [online]. 2023. [visited on 2026-05-03]. Available from: <https://www.anthropic.com/claude>.
13. GOOGLE. *Gemini* [online]. 2023. [visited on 2026-05-03]. Available from: <https://gemini.google.com>.
14. ZHANG, Zeyu; ZHANG, Xiaohe; WANG, Tao; ZHU, Dawei; WANG, Wangchunshu; CHEN, Wenhao; MA, Tianyu; ZHANG, Ruifeng. A Survey on the Memory Mechanism of Large Language Model based Agents. *arXiv preprint arXiv:2404.13501*. 2024. Available also from: <https://arxiv.org/abs/2404.13501>.
15. JSON-RPC WORKING GROUP. *JSON-RPC 2.0 Specification* [online]. 2013. [visited on 2026-03-10]. Available from: <https://www.jsonrpc.org/specification>.
16. MODEL CONTEXT PROTOCOL CONTRIBUTORS. *Model Context Protocol — GitHub Organisation* [online]. 2024. [visited on 2026-03-10]. Available from: <https://github.com/modelcontextprotocol>.
17. ANTHROPIC. *Model Context Protocol Specification* [online]. 2024. [visited on 2026-03-10]. Available from: <https://modelcontextprotocol.io/specification/latest>.
18. LEWIS, Patrick; PEREZ, Ethan; PIKTUS, Aleksandra; PETRONI, Fabio; KARPUKHIN, Vladimir; GOYAL, Naman; KÜTTLER, Heinrich; LEWIS, Mike; YIH, Wen-tau; ROCKTÄSCHEL, Tim; RIEDEL, Sebastian; KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*. 2020, vol. 33, pp. 9459–9474. Available also from: <https://arxiv.org/abs/2005.11401>.
19. CLOUD NATIVE COMPUTING FOUNDATION. *Cloud Native Computing Foundation* [online]. 2015. [visited on 2026-05-03]. Available from: <https://www.cncf.io>.

20. THE KUBERNETES AUTHORS. *Pods — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/workloads/pods/>.
21. THE KUBERNETES AUTHORS. *Namespaces — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>.
22. THE KUBERNETES AUTHORS. *Persistent Volumes — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>.
23. THE KUBERNETES AUTHORS. *StatefulSets — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/>.
24. THE KUBERNETES AUTHORS. *Custom Resources — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>.
25. THE KUBERNETES AUTHORS. *Labels and Selectors — Kubernetes Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/>.
26. RED HAT. *OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://docs.openshift.com/container-platform/latest/welcome/index.html>.
27. RED HAT. *Working with Projects — OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://docs.openshift.com/container-platform/latest/applications/projects/working-with-projects.html>.
28. RED HAT. *Managing Image Streams — OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: https://docs.openshift.com/container-platform/latest/openshift_images/image-streams-manage.html.

29. RED HAT. *Configuring Routes — OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://docs.openshift.com/container-platform/latest/networking/routes/route-configuration.html>.
30. RED HAT. *Understanding Authentication — OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://docs.openshift.com/container-platform/latest/authentication/understanding-authentication.html>.
31. RED HAT. *Understanding Operators — OpenShift Container Platform Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://docs.openshift.com/container-platform/latest/operators/understanding/olm-what-operators-are.html>.
32. KUBEFLOW CONTRIBUTORS. *Kubeflow Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://www.kubeflow.org/docs/>.
33. OPEN DATA HUB CONTRIBUTORS. *Open Data Hub Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: <https://opendatahub.io/docs/>.
34. KUBEFLOW CONTRIBUTORS. *Notebook CRD — Kubeflow GitHub Repository* [online]. 2024. [visited on 2026-03-22]. Available from: https://github.com/kubeflow/kubeflow/blob/master/components/notebook-controller/config/crd/bases/kubeflow.org_notebooks.yaml.
35. RED HAT. *Working with Workbenches — Red Hat OpenShift AI Documentation* [online]. 2024. [visited on 2026-03-22]. Available from: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/2-latest/html/working_on_data_science_projects/using-project-workbenches-workbenches.
36. GOOGLE. *The Go Programming Language* [online]. 2009. [visited on 2026-03-22]. Available from: <https://go.dev>.
37. MODEL CONTEXT PROTOCOL CONTRIBUTORS. *MCP Go SDK — GitHub Repository* [online]. 2025. [visited on 2026-03-22]. Available from: <https://github.com/modelcontextprotocol/go-sdk>.

BIBLIOGRAPHY

38. GOLANGCI-LINT CONTRIBUTORS. *golangci-lint — Fast Go linters runner* [online]. 2024. [visited on 2026-03-22]. Available from: <https://golangci-lint.run>.
39. PROMPTFOO CONTRIBUTORS. *Promptfoo: Test and evaluate LLMs* [online]. 2024. [visited on 2026-03-10]. Available from: <https://www.promptfoo.dev>.
40. CONFIDENT AI. *DeepEval — The Open-Source LLM Evaluation Framework* [online]. 2024. [visited on 2026-05-03]. Available from: <https://docs.confident-ai.com>.
41. THE KUBERNETES AUTHORS. *client-go — Go client for Kubernetes* [online]. 2024. [visited on 2026-03-22]. Available from: <https://github.com/kubernetes/client-go>.
42. MALÝ, Adam David. *MCP Server for Red Hat OpenShift AI — Source Code Repository* [online]. 2026. [visited on 2026-05-03]. Available from: <https://github.com/amaly/mcp-server-rhoai>.
43. WICKS, Mathew. *Kubeflow Notebooks 2.0 — Design CRDs* [online]. 2023. [visited on 2026-04-25]. Available from: <https://github.com/thesuperzapper/kubeflow-notebooks-v2-design/tree/main/crds>.